MANUAL TÉCNICO

# RED NEXUS

## Recolector Transaccional ISO 8583 de alto rendimiento

Arquitectura, protocolo ISO 8583, Core Bancario, API REST, seguridad, configuración, despliegue con Docker (binario nativo GraalVM sobre Distroless), operación y pruebas.

| | |
|---|---|
| Artefacto | com.banco:recolector-transaccional-iso8583:1.0.0 |
| Plataforma | Java 25 · Spring Boot 4.1.2 · Spring Framework 7 · Jakarta EE 11 |
| Integración | Spring Integration IP 7.1 (TCP) · RestClient · j8583 1.17 |
| Base de datos | Microsoft SQL Server 2022 (CoreBancario5B) |
| Imagen | GraalVM Native Image 25 → gcr.io/distroless/cc-debian12:nonroot |
| Puertos | 8087 HTTP (web, API, Actuator) · 15000 TCP (ISO 8583) |
| Fecha | Octubre de 2026 |

# Contenido

1. Descripción general
2. Arquitectura
3. Protocolo ISO 8583
4. Core Bancario y modelo de datos
5. API REST
6. Seguridad
7. Configuración
8. Despliegue
9. Operación y monitoreo
10. Pruebas y calidad
11. Solución de problemas

Documentos relacionados en el repositorio: `README.md` (instalación y uso), `DOCUMENTACION.md` (reporte técnico con decisiones de diseño y mediciones), `MANUAL_DE_USUARIO_RED_NEXUS.pdf` y `postman_collection.json`.

# 1. Descripción general

El sistema es un **switch transaccional**: recibe mensajes **ISO 8583:1987** de terminales (cajeros y POS) por un socket TCP, los traduce a llamadas REST/JSON hacia un **Core Bancario** que mantiene cuentas y transacciones en SQL Server, y responde a la terminal con el mensaje ISO correspondiente y el código de acción en el **DE 39**. La misma aplicación sirve un cajero web y un panel de administración para la demostración.

## Tecnologías

| Área | Tecnología | Uso |
|---|---|---|
| Lenguaje | Java 25 | Virtual Threads, records, pattern matching, switch expressions |
| Framework | Spring Boot 4.1.2 / Spring Framework 7.0 | Web MVC (Tomcat), Data JPA (Hibernate 7), Security 7, Actuator |
| Servidor TCP | Spring Integration IP 7.1 | `TcpNetServerConnectionFactory` + `Tcp.inboundGateway` |
| ISO 8583 | j8583 1.17.0 | Parseo y codificación según `j8583.xml` |
| Cliente HTTP | RestClient + Apache HttpClient 5 | Llamadas del Recolector al Core con `RetryTemplate` |
| Jakarta EE 11 | `jakarta.inject`, `jakarta.annotation`, `jakarta.validation`, `jakarta.persistence` | Inyección, ciclo de vida, validación, entidades |
| Base de datos | SQL Server 2022 (driver mssql-jdbc) | Cuentas y transacciones; H2 en modo MSSQL solo en pruebas |
| Imagen nativa | GraalVM Native Image 25 | Binario `recolector` con AOT de Spring |
| GC (modo JVM) | Generational ZGC | Pausas sub-milisegundo |
| Interfaz | HTML + Tailwind CSS + JavaScript | `src/main/resources/static/index.html` |

# 2. Arquitectura

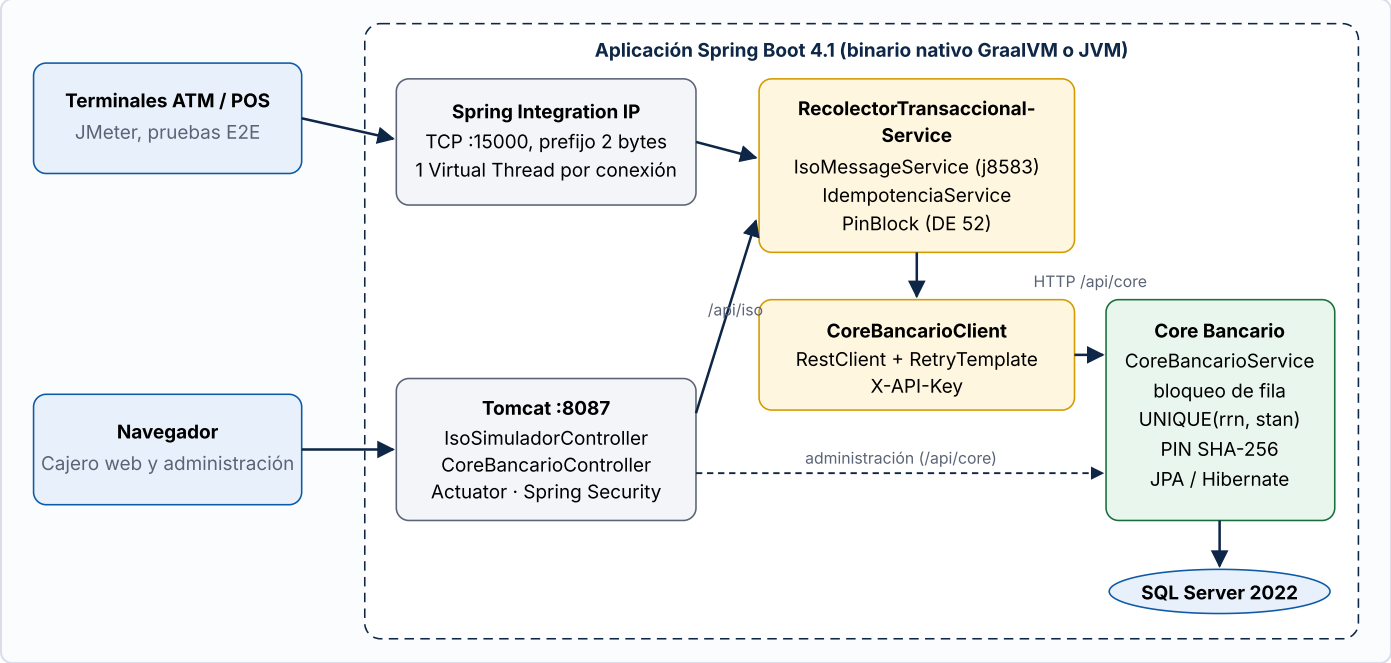



Figura 1. Componentes y flujo de una transacción.

## Componentes

| Paquete / clase | Responsabilidad |
|---|---|
| `RecolectorApplication` | Arranque; con el argumento `--healthcheck` consulta `/actuator/health` y termina con 0 (UP) o 1. |
| `config.TcpIntegrationConfig` | Servidor TCP ISO 8583: fábrica de conexiones, enmarcado de 2 bytes, gateway de entrada. |
| `config.VirtualThreadsConfig` | Ejecutor de Virtual Threads para las conexiones TCP. |
| `config.SeguridadConfig`, `ApiKeyAuthenticationFilter` | Reglas de acceso, HTTP Basic para administración y API Key para el Recolector. |
| `config.IsoAotRuntimeHints` | `RuntimeHintsRegistrar` para la imagen nativa (reflexión de j8583, `j8583.xml` y su DTD). |
| `iso.IsoMessageService` | Parseo, codificación y armado de mensajes con j8583; extracción de DE 3, 4, 11, 37, 102 y 52. |
| `iso.PinBlock` | PIN block ISO 9564 formato 0. |
| `iso.IsoResponseCode` | Catálogo de códigos DE 39. |
| `iso.controller.IsoSimuladorController` | `/api/iso`: simulador de tramas, métricas y ráfaga concurrente. |

| Paquete / clase | Responsabilidad |
|---|---|
| `service.RecolectorTransaccionalService` | Orquestación: parseo, idempotencia, llamada al Core, armado de la respuesta (DE 38, 39, 54). |
| `service.IdempotenciaService` | Caché de respuestas por RRN y STAN (retransmisiones). |
| `service.TcpIsoHealthIndicator` | Componente `tcpIso` de `/actuator/health`. |
| `corebancario.CoreBancarioClient` | RestClient hacia el Core, reintentos y traducción de fallos a DE 39. |
| `corebancario.service.CoreBancarioService` | Reglas de negocio, bloqueo pesimista, idempotencia en BD, PIN, reversas, administración. |
| `corebancario.controller.CoreBancarioController` | `/api/core`: transacciones, cuentas, administración y mapeo DE 39 → HTTP. |

## Flujo de una transacción (retiro 0200)

1. La terminal envía la trama precedida de su longitud. Spring Integration la recibe en un Virtual Thread.
2. `RecolectorTransaccionalService` la parsea con j8583. Si es ilegible responde `30` sin cerrar la conexión.
3. Si el par RRN + STAN ya fue respondido, devuelve la misma respuesta (retransmisión).
4. Si faltan STAN, RRN o cuenta, responde `30`.
5. Arma un `TransaccionCoreRequest` (record) y lo envía a `POST /api/core/transacciones` con la cabecera `X-API-Key`.
6. El Core bloquea la cuenta, valida estado, PIN, monto, límite y saldo, aplica el débito y registra la transacción en una sola transacción de BD.
7. El Recolector arma la respuesta 0210 con DE 38, DE 39 y DE 54 y guarda en la caché las respuestas definitivas (no 68, 91 ni 96).

## Concurrencia

- Cada conexión TCP tiene su propio Virtual Thread; el código es bloqueante y secuencial (socket, HTTP, JDBC) sin agotar hilos del sistema operativo.
- `spring.threads.virtual.enabled=true` aplica Virtual Threads a Tomcat y a las tareas programadas.
- No se usa `synchronized` en caminos con E/S (se usa `ReentrantLock`), para no fijar el hilo virtual a su hilo portador.
- Pool JDBC de 20 conexiones (Hikari) y pool HTTP de 500 conexiones keep-alive hacia el Core.

# 3. Protocolo ISO 8583

## 3.1 Enmarcado

- Cada trama va precedida por **2 bytes con su longitud** (entero sin signo, big-endian), sin contar esos 2 bytes. La respuesta usa el mismo enmarcado.
- Tamaño máximo de trama: 8.192 bytes ( `recolector.tcp.max-frame-bytes` ).
- La conexión puede quedar abierta para varias transacciones; se cierra tras 30 s sin actividad ( `recolector.tcp.timeout-ms` ).
- Codificación ASCII, bitmaps en hexadecimal, cabecera opcional `ISO015000050` (las respuestas la incluyen).

## 3.2 Mensajes

| Solicitud | Respuesta | Uso | Campos de la solicitud |
|---|---|---|---|
| 0200 | 0210 | Financiera: compra (DE 3 = 00xxxx), retiro (01), devolución (20), transferencia (40) | 2, 3, 4, 7, 11, 12, 13, 14, 22, 25, 37, 41, 42, 49, 52, 102, 103 |
| 0100 | 0110 | Autorización / consulta de saldo (DE 3 = 30xxxx) | 2, 3, 4, 7, 11, 12, 13, 37, 41, 42, 49, 52, 102 |
| 0420 | 0430 | Reversa: DE 37 = RRN de la transacción original; DE 11 propio | 2, 3, 4, 7, 11, 12, 13, 37, 41, 42, 49, 90, 102 |
| 0800 | 0810 | Gestión de red (DE 70 = 301, echo test) | 7, 11, 12, 13, 70 |

Las respuestas repiten los campos de la solicitud que su guía admite y agregan DE 38 (autorización, solo si se aprueba), DE 39 y DE 54 (saldos, en 00, 51 y 61). El PIN block (DE 52) nunca se devuelve. Un MTI legible pero no soportado como solicitud responde `12`.

## 3.3 Campos

| DE | Nombre | Formato | Notas |
|---|---|---|---|
| 2 | PAN | LLVAR | Si falta DE 102, la cuenta se busca por PAN |
| 3 | Código de procesamiento | n6 | Los 2 primeros dígitos definen la operación |
| 4 | Monto | n12 | Centavos (000000010000 = Q100.00) |
| 7 | Fecha y hora de transmisión | n10 | MMddHHmmss |
| 11 | STAN | n6 | Idempotencia junto con DE 37 |
| 12 / 13 | Hora / fecha local | n6 / n4 | HHmmss / MMdd |
| 37 | RRN | an12 | Referencia única; en 0420, la de la transacción original |
| 38 | Código de autorización | an6 | Solo en aprobaciones |
| 39 | Código de respuesta | an2 | Ver 3.4 |

| DE | Nombre | Formato | Notas |
|---|---|---|---|
| 41 / 42 | Terminal / comercio | an8 / an15 | |
| 49 | Moneda | n3 | 320 (GTQ) por defecto; 840 (USD) |
| 52 | PIN block | an16 (hex) | ISO 9564 formato 0 (ver 3.5) |
| 54 | Saldos | LLLVAR | Ver 3.6 |
| 70 | Código de gestión de red | n3 | 301 = echo test |
| 90 | Datos originales | an42 | Reversas |
| 102 / 103 | Cuenta origen / destino | LLVAR | Transferencias usan DE 103 |

## 3.4 Códigos de respuesta (DE 39)

| DE 39 | Significado | HTTP del Core | Origen |
|---|---|---|---|
| 00 | Aprobada | 200 | Core |
| 05 | Tarjeta bloqueada / no honrar | 403 | Core |
| 12 | Transacción inválida | 400 | Recolector o Core |
| 13 | Monto inválido | 400 | Core |
| 14 | Cuenta o tarjeta inexistente | 404 | Core |
| 25 | Transacción original no encontrada | 404 | Core (reversa) |
| 30 | Error de formato | 400 | Recolector (trama) o Core (JSON) |
| 51 | Fondos insuficientes | 422 | Core |
| 54 | Tarjeta expirada | 410 | Core |
| 55 | PIN incorrecto o ausente | 403 | Core |
| 61 | Excede el límite diario | 429 | Core |
| 68 | Tiempo de espera agotado | — | Recolector |
| 75 | Intentos de PIN excedidos (la tarjeta se bloquea) | 403 | Core |
| 91 | Emisor no disponible | 5xx o sin conexión | Recolector |
| 96 | Error del sistema | — | Recolector |

Si el Core responde con cuerpo JSON, el Recolector usa su `codigoRespuesta`. Sin cuerpo, traduce el estado HTTP: 400 → 30, 403/423 → 05, 404 → 14, 410 → 54, 422 → 51, 429 → 61, 408/504 → 68,

500/502/503 → 91, otros → 96.

### 3.5 PIN block (DE 52)

```
PIN 1234, PAN 4532000000000001
bloque PIN = 0 | longitud | PIN | relleno F                -> 041234FFFFFFFFFF
bloque PAN = 0000 | 12 dígitos del PAN sin el verificador  -> 0000200000000000
DE 52      = bloque PIN XOR bloque PAN                     -> 041214FFFFFFFFFF
```

El Core recupera el PIN con el PAN de la cuenta y lo compara en tiempo constante con `SHA-256("numeroCuenta:PIN")`. El bloque viaja sin cifrar bajo una ZPK porque el sistema no incluye un HSM.

### 3.6 Saldos (DE 54)

Dos bloques de 20 caracteres: tipo de cuenta (`10`), tipo de monto (`01` contable, `02` disponible), moneda, signo (`C` / `D`) y 12 dígitos en centavos. Ejemplo con Q1,400.00: `1001320C000000140000` + `1002320C000000140000`.

## 3.7 Ejemplo real: retiro de Q100 aprobado

Trama de solicitud (sin el prefijo de longitud):

```
ISO0150000500200F2380000088090000000000004000000164532000000000001010000000000100000102714472
000012314472010272627100001 23ATMNX001320041214FFFFFFFFFF141001-2002-3001
```

| Parte | Valor | Significado |
|---|---|---|
| Cabecera | `ISO0150000 50` | Cabecera ISO (12 caracteres) |
| MTI | `0200` | Solicitud financiera |
| Bitmaps | `F238000008809000` `0000000004000000` | Campos 1, 2, 3, 4, 7, 11, 12, 13, 37, 41, 49, 52 y 102 |
| DE 2 | `16 4532000000000001` | PAN (LLVAR) |
| DE 3 | `010000` | Retiro |
| DE 4 | `000000010000` | Q100.00 |
| DE 7 / 11 / 12 / 13 | `1027144720` `000123` `144720` `1027` | Fecha y hora, STAN, hora y fecha local |
| DE 37 | `262710000123` | RRN |
| DE 41 / 49 | `ATMNX001` `320` | Terminal y moneda |
| DE 52 | `041214FFFFFFFFFF` | PIN block |
| DE 102 | `14 1001-2002-3001` | Cuenta (LLVAR) |

Trama de respuesta:

```
ISO0150000500210F23800000E8084000000000004000000164532000000000001010000000000100000102714472
000012314472010272627100001 2310001000ATMNX0013200401001320C000000140000100 2320C00000014000014
1001-2002-3001
```

| Parte | Valor | Significado |
|---|---|---|
| MTI | `0210` | Respuesta financiera |
| Bitmap primario | `F23800000E808400` | Agrega 38, 39 y 54; sin 52 |
| DE 38 | `100010` | Código de autorización |
| DE 39 | `00` | Aprobada |
| DE 54 | `040 1001320C000000140000 1002320C000000140000` | Saldo contable y disponible: Q1,400.00 |

El simulador ( `/api/iso/simular` ) devuelve estas mismas tramas con el PAN y el PIN block enmascarados.

# 4. Core Bancario y modelo de datos

El esquema lo crea `init_sqlserver.sql` (la aplicación no modifica el esquema: `ddl-auto: none`). El script es idempotente: crea lo que falta sin borrar datos, activa `READ_COMMITTED_SNAPSHOT`, crea índices y carga las cinco cuentas de demostración; si encuentra una columna `pin` en claro la convierte a `pin_hash` y la elimina.

```
CREATE TABLE dbo.cuentas (
    id BIGINT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    numero_cuenta VARCHAR(30) NOT NULL,          -- índice único idx_cuenta_num
    pan VARCHAR(19) NOT NULL,                    -- índice idx_cuenta_pan
    titular VARCHAR(100) NOT NULL,
    saldo_disponible DECIMAL(15,2) NOT NULL,
    saldo_contable DECIMAL(15,2) NOT NULL,
    limite_diario DECIMAL(15,2) NOT NULL DEFAULT 500.00,
    retirado_hoy DECIMAL(15,2) NOT NULL DEFAULT 0.00,
    fecha_ultimo_retiro DATE NULL,
    estado VARCHAR(20) NOT NULL DEFAULT 'ACTIVE', -- ACTIVE, BLOCKED, EXPIRED
    pin_hash VARCHAR(64) NULL,                   -- SHA-256 de 'numero_cuenta:PIN' en
hexadecimal
    intentos_pin_fallidos INT NOT NULL DEFAULT 0,
    banco VARCHAR(60) NULL
);

CREATE TABLE dbo.transacciones (
    id BIGINT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    rrn VARCHAR(12) NOT NULL,                    -- UNIQUE (rrn, stan): uk_tx_rrn_stan
    stan VARCHAR(6) NOT NULL,
    numero_cuenta VARCHAR(30) NOT NULL,          -- índice idx_tx_cuenta
    cuenta_destino VARCHAR(30) NULL,
    monto DECIMAL(15,2) NOT NULL,
    tipo_operacion VARCHAR(20) NOT NULL,         -- COMPRA, RETIRO, CONSULTA, TRANSFERENCIA,
DEVOLUCION, REVERSA
    saldo_anterior DECIMAL(15,2) NULL,
    saldo_posterior DECIMAL(15,2) NULL,
    codigo_respuesta VARCHAR(2) NOT NULL,        -- DE 39
    codigo_autorizacion VARCHAR(6) NULL,         -- DE 38
    terminal_id VARCHAR(30) NULL,
    fecha_hora DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);
```

## Reglas de negocio

1. **Bloqueo:** la cuenta se lee con bloqueo pesimista (`UPDLOCK, HOLDLOCK, ROWLOCK`); en transferencias se bloquean ambas cuentas en orden de id para evitar interbloqueos.
2. **Validaciones, en orden:** cuenta existente y, si viene el PAN, que corresponda a la cuenta (14) → estado bloqueado (05) o expirado (54) → PIN, obligatorio en retiro, consulta y transferencia y

verificado siempre que venga (55; al tercer fallo 75 y la cuenta pasa a BLOCKED) → monto mayor que cero (13) → límite diario de retiro (61) → saldo (51).

3. **Aplicación:** débito o crédito de los saldos disponible y contable, acumulado diario de retiros y registro de la transacción (también las rechazadas) en la misma transacción de BD.
4. **Idempotencia:** si el par (RRN, STAN) ya existe, la inserción falla, todo se revierte y se devuelve la respuesta original.
5. **Reversa:** busca la transacción original aprobada por RRN, acredita su monto (y descuenta el acumulado diario si era retiro) una sola vez; si no existe responde 25.
6. **Autorización:** códigos secuenciales de 6 dígitos.
7. **Reset de demostración** ( `POST /api/core/reset` ): borra las transacciones, reinicia acumulados e intentos de todas las cuentas y devuelve las cinco cuentas semilla a su saldo, límite, estado y PIN iniciales. Al confirmar publica el evento `DatosRestauradosEvent` , con el que el Recolector vacía su caché de idempotencia.

# 5. API REST

Base: `http://localhost:8087`. Todas las respuestas son JSON. La colección `postman_collection.json` contiene una petición con pruebas automáticas para cada caso.

| Método | Ruta | Acceso | Descripción |
|---|---|---|---|
| POST | `/api/iso/simular` | Público | Arma una trama, la procesa con el flujo del Recolector y devuelve tramas, bitmap y DE 39 |
| GET | `/api/iso/metricas` | Público | Contadores del switch, estado del socket, memoria y GC |
| POST | `/api/iso/rafaga?cantidad=50` | Público | Ráfaga de consultas concurrentes en Virtual Threads (entre 5 y 200) |
| POST | `/api/core/transacciones` | API Key o ADMIN | Contrato Recolector → Core |
| GET | `/api/core/transacciones?limite=200` | Público | Historial (más recientes primero) |
| GET | `/api/core/cuentas` | Público | Lista de cuentas (sin PIN) |
| GET | `/api/core/cuentas/{cuenta o PAN}` | Público | Una cuenta |
| GET | `/api/core/cuentas/{id}/transacciones?limite=50` | Público | Movimientos de una cuenta |
| POST | `/api/core/cuentas` | ADMIN | Alta (201; 409 si la cuenta o el PAN existen) |
| PUT | `/api/core/cuentas/{id}` | ADMIN | Edición; `pin` omitido conserva el PIN |
| DELETE | `/api/core/cuentas/{id}` | ADMIN | Baja (204 / 404) |
| POST | `/api/core/cuentas/{id}/recargar?monto=500` | ADMIN | Depósito |
| POST | `/api/core/cuentas/{id}/toggle-bloqueo` | ADMIN | Bloquear / desbloquear |
| POST | `/api/core/reset` | ADMIN | Restaurar datos de demostración |
| GET | `/api/core/admin/sesion` | ADMIN | Verifica credenciales (login de la interfaz) |
| GET | `/actuator/health`, `/actuator/info` | Público | Salud (incluye `tcpIso` y `db`) |
| GET | `/actuator/metrics/**` | ADMIN | Métricas de Micrometer |

### Simulador: `POST /api/iso/simular`

```
{ "mti": "0200", "pan": "4532000000000001", "cuenta": "1001-2002-3001", "monto": 100.00,
  "processingCode": "010000", "terminalId": "ATMNX001", "stan": "000123", "rrn":
"262710000123",
  "moneda": "GTQ", "pin": "1234" }
```

Campos opcionales: `pan` (se busca por la cuenta), `stan` y `rrn` (se generan), `pin`, `cuentaDestino` (transferencias) y `moneda` (`GTQ` o `USD`). En una reversa 0420, `rrn` es el de la transacción original. Respuesta (resumen):

```
{ "exitosa": true, "mtiSolicitud": "0200", "mtiRespuesta": "0210",
  "codigoRespuestaIso": "00", "descripcionCodigo": "00 - Aprobado Exitosamente",
"codigoAutorizacion": "100010",
  "panEnmascarado": "453200******0001", "titular": "CARLOS PEREZ", "bancoEmisor": "BANCO
INDUSTRIAL",
  "monto": 100.0, "saldoDisponible": 1400.0, "saldoContable": 1400.0, "rrn": "262710000123",
"stan": "000123",
  "latenciaMs": 33, "bitmapHex": "F23800000880900000000000004000000",
  "bitsActivos": [1, 2, 3, 4, 7, 11, 12, 13, 37, 41, 49, 52, 102],
  "rawRequestAscii": "...", "rawResponseAscii": "...", "dataElementsEnviados": {...},
"dataElementsRecibidos": {...} }
```

### Contrato Recolector → Core: `POST /api/core/transacciones`

```
POST /api/core/transacciones
X-API-Key: nexus-core-api-key-desarrollo
{ "rrn": "262710000124", "stan": "000124", "cuentaOrigen": "1001-2002-3002", "monto": 25.00,
  "tipoOperacion": "COMPRA", "terminalId": "POS00001", "codigoProcesamiento": "000000" }

HTTP 200
{ "exitosa": true, "codigoRespuesta": "00", "codigoAutorizacion": "100011",
"saldoDisponible": 325.00,
  "saldoContable": 325.00, "mensaje": "Transacción Aprobada Exitosamente", "rrn":
"262710000124", ... }

HTTP 422 (otra transacción, cuenta con Q10.00 y monto Q50.00)
{ "exitosa": false, "codigoRespuesta": "51", "codigoAutorizacion": null, "saldoDisponible":
10.00,
  "mensaje": "Fondos insuficientes", "rrn": "262710000125", ... }
```

Campos: `rrn` (obligatorio, máx. 12), `stan` (obligatorio, máx. 6), `cuentaOrigen` o `pan` (uno obligatorio), `cuentaDestino`, `monto`, `tipoOperacion` (COMPRA, RETIRO, DEVOLUCION, CONSULTA, TRANSFERENCIA, REVERSA), `terminalId`, `codigoProcesamiento` y `pinBlock` (16 hexadecimales). Un cuerpo inválido responde 400 con `codigoRespuesta` = `30`.

### Errores de administración

```
POST /api/core/cuentas  (número de cuenta repetido)   -> 409 {"error":"Ya existe una cuenta
con el número: 1001-2002-3001"}
POST /api/core/cuentas  (pin "12")                    -> 400 {"error":"El PIN debe tener
exactamente 4 dígitos"}
Cualquier ruta ADMIN sin credenciales válidas          -> 401 (sin cabecera WWW-Authenticate)
```

## 6. Seguridad

| Elemento | Control |
|---|---|
| Transacciones del Core | Solo el Recolector (cabecera `X-API-Key`, comparación en tiempo constante) o un administrador. |
| Administración y métricas | Rol ADMIN con HTTP Basic. Usuario y contraseña por variables de entorno; contraseña con BCrypt, calculada en el primer login. |
| Consultas y simulador | Públicos (no exponen PIN ni hash). |
| PIN | Hash SHA-256 en BD; PIN block en la trama; eliminado de respuestas y enmascarado en el simulador; 3 intentos fallidos bloquean la tarjeta. |
| Sesión | API sin estado (sin cookies), por eso CSRF no aplica. La interfaz guarda las credenciales solo en memoria de la pestaña y cierra la sesión ante un 401. |
| Trama TCP | Tamaño máximo de 8 KB; tramas ilegibles responden 30 sin afectar la conexión. |
| Contenedor | Distroless sin shell ni gestor de paquetes, usuario `nonroot` (65532), solo el binario. |

> Para producción faltaría: cifrar el PIN block bajo una ZPK y verificarlo en un HSM, TLS (o mTLS) en el socket TCP y en HTTP, y gestionar las credenciales con un almacén de secretos en lugar de `.env`.

# 7. Configuración

## 7.1 `application.yaml`

| Propiedad | Valor por defecto | Descripción |
|---|---|---|
| `server.port` | 8087 | Puerto HTTP (variable `SERVER_PORT`) |
| `spring.datasource.url` | jdbc:sqlserver://127.0.0.1:1434; databaseName=CoreBancario5B; ... | Conexión (variable `SPRING_DATASOURCE_URL`); incluye `sendStringParametersAsUnicode=false` |
| `spring.datasource.username / password` | sa / admin | Credenciales (variables `SPRING_DATASOURCE_USERNAME` / `_PASSWORD`) |
| `spring.datasource.hikari.maximum-pool-size` | 20 | Conexiones JDBC |
| `spring.threads.virtual.enabled` | true | Virtual Threads en Tomcat y tareas |
| `recolector.tcp.port` | 15000 | Puerto del socket ISO 8583 |
| `recolector.tcp.timeout-ms` | 30000 | Cierre de conexiones inactivas |
| `recolector.tcp.max-frame-bytes` | 8192 | Tamaño máximo de trama |
| `recolector.tcp.backlog` | 2048 | Cola de conexiones pendientes |
| `recolector.iso.moneda` | 320 | Moneda por defecto (DE 49) |
| `recolector.corebancario.url` | http://localhost:${server.port}/api/core | URL del Core |
| `recolector.corebancario.connect-timeout-ms / read-timeout-ms` | 2000 / 5000 | Tiempos de espera hacia el Core |
| `recolector.corebancario.reintentos / espera-reintento-ms` | 2 / 100 | Reintentos con espera exponencial |
| `recolector.seguridad.api-key` | ${CORE_API_KEY} | API Key del Recolector |
| `recolector.seguridad.admin-usuario / admin-clave` | ${ADMIN_USER} / ${ADMIN_PASSWORD} | Administrador |
| `recolector.idempotencia.ttl-minutes / max-entries` | 60 / 50000 | Caché de respuestas por RRN y STAN |

## 7.2 Variables de `.env` (Docker Compose)

| Variable | Por defecto | Uso |
|---|---|---|
| `MSSQL_SA_PASSWORD` | Nexus2026_Pass | Contraseña de `sa` (se fija al crear el volumen) |
| `APP_DOCKERFILE` | Dockerfile | `Dockerfile` (nativo) o `Dockerfile.jvm` |
| `ADMIN_USER` / `ADMIN_PASSWORD` | admin / Nexus2026_Admin | Administrador |
| `CORE_API_KEY` | nexus-core-api-key-desarrollo | API Key del Recolector |
| `APP_HTTP_PORT` / `APP_TCP_PORT` / `SQLSERVER_PORT` | 8087 / 15000 / 14333 | Puertos publicados |
| `CARGA_TERMINALES` / `CARGA_DURACION` | 200 / 60 | Prueba de carga |
| `TZ` | America/Guatemala | Zona horaria de los contenedores |

# 8. Despliegue

## 8.1 Docker Compose (recomendado)

Requisitos: Docker Desktop (WSL 2 en Windows) con al menos 8 GB de memoria para la compilación nativa.

```
docker compose up -d --build                      # SQL Server + creación de la BD + aplicación
docker compose ps                                 # cajero-app debe quedar "healthy"
docker compose logs -f app                        # registros (incluye el tiempo de arranque)
docker compose --profile tests run --rm --build tests     # pruebas + cobertura en reportes/jacoco
docker compose --profile carga run --rm jmeter    # prueba de carga, reporte en reportes/jmeter/html
docker compose --profile demo run --rm simulador retiro   # trama ISO 8583 real por TCP
docker compose --profile comparacion up -d --build app-jvm   # la misma app sobre la JVM
docker compose down                               # detener (conserva datos)
docker compose down -v                            # detener y borrar la base de datos
```

| Servicio | Imagen | Función |
|---|---|---|
| `sqlserver` | mcr.microsoft.com/mssql/server:2022-latest | Base de datos (volumen `sqlserver-data`) |
| `db-init` | la misma | Crea `CoreBancario5B` y ejecuta `init_sqlserver.sql`; termina con código 0 |
| `app` | construida con `Dockerfile` o `Dockerfile.jvm` | Aplicación; arranca cuando `db-init` terminó bien |
| `tests` (perfil) | etapa `test` del Dockerfile | `mvn verify` sobre H2 |
| `jmeter` (perfil) | `Dockerfile.jmeter` | JMeter 5.6.3 sobre Java 21 |
| `simulador` (perfil `demo`) | maven:3.9-eclipse-temurin-25 | Ejecuta `simulador/SimuladorIso.java`: terminal ISO 8583 por TCP |
| `app-jvm` (perfil `comparacion`) | `Dockerfile.jvm` | La misma aplicación sobre la JVM con ZGC (puertos 8088 y 15001) y registro de las pausas de GC |

## 8.2 Imágenes

- **`Dockerfile` (nativo):** etapa `deps` sobre `ghcr.io/graalvm/native-image-community:25` con Maven; etapa `build` con `mvn -Pnative -DskipTests package`; imagen final `gcr.io/distroless/cc-debian12:nonroot` con solo `/app/recolector` y `HEALTHCHECK ["/app/recolector", "--healthcheck"]`. Compilación de 5-10 minutos.
- **`Dockerfile.jvm`:** Maven + Temurin 25 → `eclipse-temurin:25-jre` con `-XX:+UseZGC -XX:MaxRAMPercentage=75`, usuario sin privilegios y el mismo modo `--healthcheck`. Compilación de unos 2 minutos.

- Opciones de la imagen nativa (perfil `native` del `pom.xml`): `--static-nolibc`, `-march=compatibility`, `-H:+AddAllCharsets` (necesario para el driver de SQL Server) y `--enable-monitoring=heapdump,jfr`.

### 8.3 Ejecución local sin Docker

Requisitos: JDK 25, Maven 3.9 y SQL Server con la base creada por `init_sqlserver.sql`.

```
mvn spring-boot:run                        # JVM (conexión por defecto 127.0.0.1:1434,
sa/admin)
mvn verify                                 # pruebas y verificación de cobertura (H2, sin SQL
Server)
mvn -Pnative -DskipTests package           # binario nativo target/recolector (GraalVM 25)
```

## 9. Operación y monitoreo

### Salud

```
GET /actuator/health
{"status":"UP","components":{"db":{"status":"UP"},"tcpIso":{"status":"UP"},"diskSpace":
{...},"ping":{...},...}}
```

Con credenciales de administrador, `tcpIso` incluye el puerto y las conexiones abiertas. El contenedor usa el mismo endpoint en su HEALTHCHECK.

### Métricas del switch

```
GET /api/iso/metricas
{"transaccionesProcesadas":5,"transaccionesAprobadas":4,"transaccionesRechazadas":1,"tramasCo
nErrorFormato":0,
 "tcpPuerto":15000,"tcpEstado":"ACTIVO (Listening)","tcpConexionesAbiertas":0,
 "modoConcurrencia":"Virtual Threads (Project Loom)","recolectorBasura":"ZGC
...","memoriaRssMb":964,...}
```

`/actuator/metrics` (ADMIN) expone las métricas de Micrometer; en modo JVM, `jvm.gc.pause` muestra las pausas de ZGC. El binario nativo usa Serial GC y no publica esa métrica.

### Registros

- Arranque: `Started RecolectorApplication in X seconds` y un resumen con los puertos.
- Advertencias de negocio: tramas ilegibles, MTI no soportados, bloqueo de tarjeta por intentos de PIN, Core no disponible.
- Nivel por paquete: `logging.level.com.banco.recolector` (por defecto INFO; DEBUG muestra cada trama sin datos sensibles).

## Datos de demostración

`POST /api/core/reset` (o **Restaurar datos** en el panel) devuelve las cuentas semilla a su estado inicial y vacía la caché de idempotencia del Recolector. `docker compose down -v` borra la base completa; el siguiente `up` la recrea.

# 10. Pruebas y calidad

119 pruebas (JUnit 6, AssertJ, Mockito, MockMvc, WireMock 3) se ejecutan con `mvn verify` sobre H2 en modo MSSQL. JaCoCo falla el build si la cobertura de líneas baja de 80 %; la última ejecución dio 96,8 % de líneas y 84,5 % de ramas.

| Clase | Alcance |
|---|---|
| `IsoParserUnitTest` | j8583: codificación, parseo, bitmaps, DE 54, tramas corruptas |
| `PinBlockTest`, `IsoResponseCodeTest`, `TransaccionCoreRequestTest` | PIN block, catálogo DE 39, tipo de operación |
| `WireMockCoreBancarioIntegrationTest` | RestClient: estados HTTP, reintentos, timeouts, caídas, API Key |
| `RecolectorTransaccionalServiceTest`, `IdempotenciaServiceTest` | Orquestación e idempotencia |
| `CoreBancarioServiceIntegrationTest` | Reglas de negocio y concurrencia real |
| `CoreBancarioControllerIntegrationTest`, `IsoSimuladorControllerIntegrationTest` | API, seguridad, validaciones, reset |
| `RecolectorEndToEndTest` | Socket TCP → Recolector → Core → BD |
| `LoadTestSimulationTest` | Carga por TCP con JFR: pausas de ZGC ≤ 1 ms y sin pinning (reporte en `target/site/reporte-carga-jfr.txt`; con Docker, `reportes/reporte-carga-jfr.txt`) |
| `IsoAotRuntimeHintsTest` | RuntimeHints de la imagen nativa |

## Prueba de carga

`src/test/resources/load-test-iso8583.jmx` crea una cuenta por terminal, envía retiros 0200 con PIN block por conexiones TCP persistentes verificando 0210 con DE 39 = 00, y al terminar borra esas cuentas. Se ejecuta con `docker compose --profile carga run --rm jmeter` o con un JMeter local sobre Java 17/21. Los resultados medidos y su metodología están en `DOCUMENTACION.md`; cómo ejecutar cada prueba y leer sus métricas, en `PRUEBAS.md`, y cómo compilar con y sin Docker, en `COMPILACION.md`.

# 11. Solución de problemas

| Síntoma | Causa | Acción |
|---|---|---|
| `cajero-app` no pasa a *healthy* | La BD no está lista o las credenciales no coinciden. | `docker compose logs app`; revisar `MSSQL_SA_PASSWORD`; si se cambió después de crear el volumen, `docker compose down -v`. |
| `cajero-db-init` termina con error | SQL Server no aceptó la conexión o el script falló. | `docker compose logs db-init`. |
| La compilación nativa se interrumpe | Memoria insuficiente en Docker. | Asignar 8 GB o más a WSL 2, o usar `APP_DOCKERFILE=Dockerfile.jvm`. |
| "Port is already allocated" | Puerto ocupado en el equipo. | Cambiar `APP_HTTP_PORT`, `APP_TCP_PORT` o `SQLSERVER_PORT` en `.env`. |
| Todas las transacciones responden 91 | El Recolector no alcanza al Core. | Revisar `recolector.corebancario.url` y que la app escuche en ese puerto. |
| Todas las transacciones responden 96 | La API Key del Recolector no coincide con la del Core (el Core responde 401). | Usar el mismo `CORE_API_KEY`. |
| El binario nativo no conecta a SQL Server (codepage Cp1252) | Imagen compilada sin todos los charsets. | Compilar con el perfil `native` del `pom.xml` (incluye `-H:+AddAllCharsets`). |
| Los scripts de JMeter fallan con "Unsupported class file major version 69" | JMeter 5.6.3 ejecutado con Java 25. | Usar el servicio `jmeter` de Compose o Java 17/21 para JMeter. |
| 401 en operaciones de administración | Credenciales distintas a `ADMIN_USER` / `ADMIN_PASSWORD`. | Revisar `.env` y reiniciar la app. |